

Deployment

Building a static Almanac site, setting site and base, and the things CI needs to get right.

Shravan Goswami / <https://shravangoswami.com>

An Almanac site is an ordinary static Astro build. `npm run build` writes HTML, CSS, JS, the OG image, `robots.txt`, and the Pagefind index into `dist/`, and any static host will serve that directory as-is. There is no server component and no deploy step inside the framework.

site and base

Set both in `astro.config.mjs`, not in the Almanac options:

```
export default defineConfig({
  site: "https://example.com",
  base: process.env.BASE_PATH || "/my-docs",
  integrations: [almanac({ title: "My Project" })],
});
```

`site` is the absolute origin. Canonical links, Open Graph URLs, the RSS feed, and the Sitemap: line in `robots.txt` are all built from it, so a missing or wrong `site` produces links that point at the wrong place.

`base` is the subpath the site is served under, which you need for project-level GitHub Pages. Almanac prefixes every link it generates with the `base`, so the only thing you have to remember is to keep links inside your Markdown relative (`../guides/search/`, not `/docs/guides/search/`). See [Writing docs](#).

Reading `base` from an environment variable is what makes per-PR previews possible: build the same tree with `BASE_PATH=/my-docs/pr-123` and the preview links resolve inside that subfolder instead of colliding with production.

GitHub Pages

```
# .github/workflows/deploy.yml
name: Deploy
on:
  push:
    branches: [main]
permissions:
  contents: read
  pages: write
  id-token: write
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: 22
          cache: npm
      - run: npm ci
      - run: npm run build
      env:
        BASE_PATH: /my-docs
      - uses: actions/upload-pages-artifact@v3
        with:
          path: dist
  deploy:
```

```
needs: build
runs-on: ubuntu-latest
environment:
  name: github-pages
steps:
  - uses: actions/deploy-pages@v4
```

Netlify, Vercel, and Cloudflare Pages need the same two things: `npm run build` as the command and `dist` as the output directory. Drop `base` entirely when the site is served from a domain root.

What CI has to get right

Two build steps reach outside your repository, and both fail soft, which means a broken deploy looks like a missing feature rather than a red build:

- **Pagefind.** The integration spawns `npx -y pagefind --site dist` after the build. Keep `pagefind` in your `devDependencies` so `npx` resolves the local copy instead of downloading one. If it can't run, the build logs `search index skipped` and finishes, and you ship a site whose search box returns nothing.
- **The OG image.** Satori fetches the Inter font from Google Fonts at build time. Without network access it falls back to a generic sans-serif, so the image still renders, just not in the right typeface.

If either matters to you, grep the build log rather than trusting a green check.

Sitemap

Almanac generates `robots.txt` and points its `Sitemap:` line at `/sitemap-index.xml`, but it does not generate the sitemap itself. Add Astro's official integration alongside it:

```
import sitemap from "@astrojs/sitemap";

integrations: [sitemap(), almanac({ title: "My Project" })],
```

Analytics

Umami is supported and off unless you opt in. Two build-time environment variables drive it:

- `PUBLIC_UMAMI_WEBSITE_ID`: your website id. Empty means no script tag is rendered at all.
- `PUBLIC_UMAMI_SRC`: the script URL, defaulting to `https://cloud.umami.is/script.js` for self-hosted instances.

Set them as repository variables and pass them into the build step alongside `BASE_PATH`. Nothing is committed to the repo and nothing is added to the page without the id.